

Perspectiva computacional de las ELM

Pragmatismo



Computación y ELM.

- Se considera 2 scripts que ilustran el desarrollo del algoritmo ELM desde una perspectiva computacional
- Se describe por bloques las actividades que realizan los mencionados scripts



Computación y ELM.

- Se utiliza, como **ejemplo 1**, una base de datos etiquetada denominada **glass_dataset** con nueve atributos y dos etiquetas nominales
- Los datos se obtienen del repositorio de aprendizaje automático UCI y constan de 214 muestras de fragmento de vidrio que pueden contener los elementos señalados en la figura

Atributos DB glass_dataset			
1.	Índice de refracción	6.	Potasio
2.	Sodio	7.	Calcio
3.	Magnesio	8.	Bario
4.	Aluminio	9.	Hierro
5.	Silicio	Etiquetas o clases	1. Window glass 2. Non-window glass

Computación y ELM.

- Llamado de los datos y normalización de la DB



```
D=load('glass_dataset');  
Z=D.glassInputs(1:9,1:214);  
Vmax=max(max(Z));  
GlassNor=Z/Vmax;  
G=GlassNor;
```

Computación y ELM.

- Creación de los conjuntos de entrenamiento (**X**) y prueba (**XX**) y asignación de etiquetas para **X** (**Y**) y para **XX** (**YY**)



```
X=GlassNor(1:9, 64:214);  
Y=D.glassTargets(1:2, 64:214);  
XX=GlassNor(1:9, 1:63);  
YY=D.glassTargets(1:2, 1:63);
```

Computación y ELM.

- Asignación heurística del número de neuronas ocultas y llamado de la función principal



```
NumNeuronas=50;% Método heurístico  
[EntSensibilidad,ValSensibilidad,pesos_iniciales,bias_iniciales]=PruebaGeninv(X',Y',XX',YY',NumNeuronas,7);
```


Computación y ELM.

- Asignación aleatoria de los pesos (**pesos_iniciales**) y de la matriz de sesgos (**BiasM**) que resuelve el problema algebraico de incompatibilidad



```
input_weights=rand(Neuronas,size(X,2));  
pesos_iniciales=input_weights;  
%Parametros capa oculta  
Bias=rand(Neuronas,1);  
bias_iniciales=Bias;  
ind=ones(1,size(X,1));  
BiasM=Bias(:,ind);
```


Computación y ELM.

```
tempH=input_weights.X'+BiasM;  
%%%%%%%%%% Calculate hidden neuron output matrix H  
switch lower(ActivationFunction)  
case {'sig','sigmoid'}  
    %%%%%%%%% Sigmoid  
    H = 1 ./ (1 + exp(-tempH));  
case {'sin','sine'}  
    %%%%%%%%% Sine  
    H = sin(tempH);  
case {'hardlim'}  
    %%%%%%%%% Hard Limit  
    H = double(hardlim(tempH));  
case {'tribas'}  
    %%%%%%%%% Triangular basis function  
    H = tribas(tempH);  
case {'radbas'}  
    %%%%%%%%% Radial basis function  
    H = radbas(tempH);  
    %%%%%%%%% More activation functions can be added here  
end
```



Cálculo de la matriz H

Computación y ELM.

- Cálculo de la matriz de pesos de la capa de salida (**B**) y de las etiquetas automáticas de entrenamiento (**Y**).



```
% Calculate output weights  
OutputWeight (beta_i)  
B=pinv(H') * T';  
% Calculate output of the  
training data  
Y=(H' * B) ';
```

Computación y ELM.

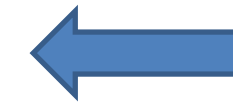
```
%Calculate training classification accuracy
if Elm_Type == CLASSIFIER

    MissClassificationRate_Training=0;
    MissClassificationRate_Testing=0;

    for i = 1 : size(T, 2)
        [x, label_index_expected]=max(T(:,i));
        [x, label_index_actual]=max(Y(:,i));
        if label_index_actual~=label_index_expected

            MissClassificationRate_Training=MissClassificationRate_Training+1;
        end
    end

    TrainingAccuracy=1 - MissClassificationRate_Training/size(T,2)
```



Cálculo de la precisión
del entrenamiento de la
ELM (TrainingAccuracy)

Computación y ELM.

- Cálculo de la precisión durante la prueba de la ELM (TestingAccuracy)

```
%Calculate testing classification accuracy
for i = 1 : size(TV.T, 2)
    [x, label_index_expected]=max(TV.T(:,i));
    [x, label_index_actual]=max(TY(:,i));
    if label_index_actual~=label_index_expected
        MissClassificationRate_Testing=MissClassificationRate_Testing+1;
    end
end
TestingAccuracy =1 - MissClassificationRate_Testing/size(TV.T,2)
end
```



Computación y ELM.

Resultado del cálculo de la precisión del entrenamiento y de la prueba de la ELM usando la DB glass_dataset



```
Tasa_acierto_entrenamiento =
```

```
0.9536
```

```
Tasa_acierto_validacion =
```

```
0.9048
```

Computación y ELM.

- Se utiliza, como **ejemplo 2**, una base de datos etiquetada denominada **cancer_dataset** con nueve atributos y dos etiquetas nominales (**benigno** y **maligno**)
- Los datos provienen del hospital de la Universidad de Wisconsin y corresponden a 699 pacientes con tumores, de los cuales 458 son benignos y 241 son malignos.

Atributos			
1.	Grosor del clúster de células (tumor)	6.	Núcleo desnudo
2.	Uniformidad del tamaño de la célula	7.	Cromatina suave
3.	Uniformidad de la forma de la célula	8	Nucléolo normal
4.	Adhesión marginal	9.	Mitosis (división de la célula)
5.	Tamaño de la célula del tejido sencillo	Clases	1 Benigno 2 Maligno

Computación y ELM.

- Se utiliza, como ejemplo, una base de datos etiquetada denominada **cancer_dataset** con nueve atributos y dos etiquetas nominales, convertidas en numéricas (0 y 1).



 2x699 double

	1	2	3	4	5	6	7	8	9	10
1	1	1	1	0	1	1	0	0	0	1
2	0	0	0	1	0	0	1	1	1	0

Computación y ELM.

- Se utiliza, como ejemplo, una base de datos etiquetada denominada **cancer_dataset** con nueve atributos, que ya viene normalizada entre 0 y 1



 9x699 double

	1	2	3	4	5	6	7	8	9	10
1	0.2000	0.2000	0.5000	0.5000	0.5000	0.2000	0.3000	1	1	0.4000
2	0.1000	0.1000	0.1000	0.4000	0.3000	0.3000	0.5000	0.5000	0.9000	0.1000
3	0.1000	0.1000	0.1000	0.6000	0.3000	0.1000	0.7000	0.6000	0.8000	0.1000
4	0.1000	0.1000	0.1000	0.8000	0.1000	0.1000	0.8000	1	0.7000	0.1000
5	0.2000	0.2000	0.2000	0.4000	0.2000	0.3000	0.8000	0.6000	0.6000	0.2000
6	0.1000	0.1000	0.1000	0.1000	0.1000	0.1000	0.9000	1	0.4000	0.1000
7	0.2000	0.3000	0.2000	0.8000	0.2000	0.1000	0.7000	0.7000	0.7000	0.3000
8	0.1000	0.1000	0.1000	1	0.1000	0.1000	1	0.7000	1	0.1000
9	0.1000	0.1000	0.1000	0.1000	0.1000	0.1000	0.7000	1	0.3000	0.1000

Computación y ELM.

- Llamado de la DB, creación de los conjuntos tanto de entrenamiento (**X**) como de prueba (**XX**) y asignación de etiquetas para **X** (**Y**) y para **XX** (**YY**)



```
D=load('cancer_dataset');  
X=D.cancerInputs(1:9,1:490);  
Y=D.cancerTargets(1:2,1:490);  
XX=D.cancerInputs(1:9,491:699);  
YY=D.cancerTargets(1:2,491:699);
```

Computación y ELM.

- Asignación heurística del número de neuronas ocultas y llamado de la función principal



```
NumNeuronas=10;% Método heurístico  
[EntSensibilidad,ValSensibilidad,pesos_iniciales,bias_iniciales,BiasM]=PruebaGeninv(X',Y',XX',YY',NumNeuronas,7);
```

Computación y ELM.

- Asignación aleatoria de los pesos (**pesos_iniciales**) y de la matriz de sesgos (**BiasM**) que resuelve el problema algebraico de incompatibilidad



```
input_weights=rand(Neuronas,size(X,2));  
pesos_iniciales=input_weights;  
%Parametros capa oculta  
Bias=rand(Neuronas,1);  
bias_iniciales=Bias;  
ind=ones(1,size(X,1));  
BiasM=Bias(:,ind);
```

Computación y ELM.

- Cálculo de la matriz H , la matriz de pesos de la capa de salida (B) y de las etiquetas de entrenamiento (Y).



```
% Calculate H matrix
```

```
H=sig(tempH);
```

```
%Calculate output weights
```

```
OutputWeight (beta_i)
```

```
B=pinv(H') * T';
```

```
% Calculate output of the  
training data
```

```
Y=(H' * B)';
```


Computación y ELM.

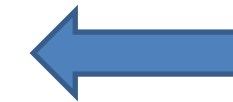
```
%Calculate training classification accuracy
if Elm_Type == CLASSIFIER

    MissClassificationRate_Training=0;
    MissClassificationRate_Testing=0;

    for i = 1 : size(T, 2)
        [x, label_index_expected]=max(T(:,i));
        [x, label_index_actual]=max(Y(:,i));
        if label_index_actual~=label_index_expected

            MissClassificationRate_Training=MissClassificationRate_Training+1;
        end
    end

    TrainingAccuracy=1 - MissClassificationRate_Training/size(T,2)
```



Cálculo de la precisión
del entrenamiento de la
ELM (TrainingAccuracy)

Computación y ELM.

- Cálculo de la precisión durante la prueba de la ELM (TestingAccuracy)

```
%Calculate testing classification accuracy
for i = 1 : size(TV.T, 2)
    [x, label_index_expected]=max(TV.T(:,i));
    [x, label_index_actual]=max(TY(:,i));
    if label_index_actual~=label_index_expected
        MissClassificationRate_Testing=MissClassificationRate_Testing+1;
    end
end
TestingAccuracy =1 - MissClassificationRate_Testing/size(TV.T,2)
end
```



Computación y ELM.

Resultado del cálculo de la precisión del entrenamiento y de la prueba de la ELM usando la DB cancer_dataset



```
Tasa_acierto_entrenamiento =
```

```
0.9612
```

```
Tasa_acierto_validacion =
```

```
0.9665
```

Gracias

En resumen:

Las ELM se implementan computacionalmente con una sencillez sorprendente debido a la simplicidad teórica de su algoritmo.

Miguel Vera

Bibliografía

- [1] Fisher R. (1936) The use of multiple measurements in taxonomic problems. Annual Eugenics, 7, Part II, 179-188
- [2] Huang G., Zhu Q., Siew C. (2006) Extreme learning machine: Theory and applications, Neurocomputing, vol. 70, nos. 1_3, pp. 489_501
- [3] Minguillón J y Casas J. (2019.) Minería de datos: Modelos y algoritmos. Barcelona: Editorial UOC, Disponible en: <https://ezproxy.unisimon.edu.co:2258/es/ereader/unisimon/58656>